

SecDash

Dashboard de Vulnerabilidades de Segurança Multi-Repositório

Gonçalo Filipe Domingos Carvalho
Rodrigo dos Ramos Pais Henriques Vitorino

Orientador: Professor Doutor José Simão

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

SecDash

49219 Gonalo Filipe Domingos Carvalho

49448 Rodrigo dos Ramos Pais Henriques Vitorino

Orientadores: Professor Doutor Jos  Sim o

Relat rio do projeto realizado no  mbito de Projecto e Semin rio
Licenciatura em Engenharia Inform tica e de Computadores

Junho de 2026

Resumo

SecDash é uma plataforma web de monitorização de relatórios de segurança que agrega e centraliza vulnerabilidade indentificadas por ferramentas externas, nomeadamente GitHub Dependabot e Code Scanning e GitLab SAST e Dependency Scanning, em diferentes repositórios.

A aplicação pretende fornecer aos seus utilizadores uma visão unificada das análises de segurança efectuadas pelas ferramentas indicadas acima, eliminando a necessidade de consultar cada plataforma e respetivos repositórios individualmente.

O desenvolvimento deste projeto utilizou como base React e Typescript no frontend, Kotlin e Spring Boot no backend e PostgreSQL para a base de dados. A autenticação dos utilizadores é feita com base no protocolo OAuth2.

Utilização de Inteligência Artificial

Neste trabalho foram utilizadas ferramentas de inteligência artificial generativa, designadamente chatbots LLM como ChatGPT, Gemini e Claude Code para clarificação de conceitos teóricos e otimização de sintaxe.

Índice

1	Introdução	1
1.1	Contextualização do problema	1
1.2	Enquadramento e estado da arte	2
1.2.1	DefectDojo	3
1.2.2	ArmorCode	4
1.2.3	ArcherySec	4
1.2.4	SecObserve	4
1.3	Requisitos Funcionais	4
2	Arquitetura e tecnologias	5
2.1	Visão geral do sistema	5
2.2	APIs externas	5
2.3	Tecnologias e linguagens	5
2.3.1	Backend	5
2.3.2	Frontend	5
3	Implementação	7
3.1	Modelo de dados	7
3.2	Backend	7
3.3	Frontend	7
4	Deployment	9
5	Dificuldades Técnicas e teóricas	11
6	Conclusão	13
	Referências	16
A	Exemplo de apêndice	17

Capítulo 1

Introdução

1.1 Contextualização do problema

O desenvolvimento de software moderno frequentemente utiliza bibliotecas *open source* para aumentar a velocidade de desenvolvimento e facilitar a implementação de componentes e funcionalidades comuns. A utilização destas bibliotecas de terceiros muitas vezes acaba por ser uma das características fundamentais do desenvolvimento de software atual, permitindo «obter um menor *time-to-market* e uma melhor qualidade do sistema de software».[1]

Contudo, a utilização destes componentes de código aberto pode introduzir vulnerabilidades de segurança, o que tem resultado em diversos incidentes de alto perfil nos últimos anos. [2] De entre os exemplos mais críticos e mediáticos deste tipo de incidentes nos últimos anos contam-se o *Heartbleed*[3] ou o *Log4Shell*[4]. Este primeiro consistiu numa falha crítica na biblioteca OpenSSL, amplamente utilizada para implementar o protocolo HTTPS por vários websites. Este incidente foi particularmente grave uma vez que resultou na exposição de chaves privadas e outros dados de alta sensibilidade. Para além disso tratava-se de uma vulnerabilidade de exploração simples e que não deixava rasto, tendo o facto de ter permanecido indetetada durante um longo período amplificado significativamente o seu impacto global.

Por outro lado, o incidente *Log4Shell*, identificado no final de 2021, afetou a biblioteca de *logging* Java **Apache Log4j**. Esta vulnerabilidade foi classificada com a pontuação máxima de severidade (10.0 no sistema CVSS), dado que permitia a execução remota de código de forma trivial através da manipulação de mensagens de log. Devido à integração desta biblioteca em milhares de serviços empresariais, a magnitude desta vulnerabilidade foi amplamente reconhecida pela indústria: a empresa de cibersegurança Tenable classificou-a como «a maior e mais crítica vulnerabilidade de sempre»[5], enquanto a publicação Ars Technica a descreveu como, «discutivelmente a vulnerabilidade mais grave de sempre»[6]. Por sua vez, o The Washington Post salientou que as descrições feitas por profissionais de cibersegurança sobre o impacto do incidente «roçam o apocalíptico»[7].

O caso da Equifax, empresa considerada um dos três maiores bureaus de crédito dos Estados Unidos da América, ilustra também o problema. Uma auditoria interna em 2015 revelou sérios problemas de organização no seu departamento de TI. O relatório elaborado com base nesta auditoria concluiu que a organização não cumpria os seus próprios calendários de *patching*, que a equipa carecia de um inventário de recursos abrangente e que não existia uma priorização dos *patches* a serem efetuados com base na criticidade. Embora o relatório tenha delineado medidas para reforçar a segurança, muitas destas não foram implementadas atempadamente[8]. Dois anos mais tarde, em 2017, os sistemas informáticos da Equifax foram invadidos com recurso a uma vulnerabilidade na *framework* Apache Struts já conhecida e corrigida meses antes. O ataque, que podia ter sido facilmente evitado com uma melhor organização operacional, resultou na exposição de dados pessoais de mais de 150 milhões de pessoas, de entre os quais alguns de alta sensibilidade tais como números de segurança social e de cartões de crédito[9].

Ainda assim, o uso destas bibliotecas de código aberto tem vindo a crescer. Como tal, várias ferramentas de análise e *pipelines* CI/CD foram desenvolvidas. As equipas de desenvolvimento de software modernas dependem muitas vezes da integração destas ferramentas de *scanning* nas suas *pipelines* CI/CD tais como GitHub Dependabot, GitLab SAST, Trivy ou OWASP Dependency-Check. Apesar de estas ferramentas serem eficazes na deteção de vulnerabilidades conhecidas, os seus resultados ficam dispersos por diferentes repositórios e plataformas, tornando-se difícil obter uma visão unificada e consolidada da postura de segurança global de uma organização.

O **SecDash** pretende ser uma plataforma web concebida para agregar a visualização de vulnerabilidades de segurança já identificadas por estas ferramentas externas ou por pipelines de CI/CD existentes. Em vez de realizar a sua própria análise, o SecDash recolhe relatórios de vulnerabilidades de múltiplos repositórios de código-fonte e apresenta-os através de um *dashboard* único e interativo, permitindo que arquitetos de segurança de software e responsáveis de desenvolvimento monitorizem o risco em todos os seus projetos de forma imediata.

1.2 Enquadramento e estado da arte

Perante o cenário histórico de vulnerabilidades críticas como as anteriormente mencionadas, a necessidade de ferramentas de agregação torna-se evidente, existindo atualmente várias soluções amplamente utilizadas na indústria. Contudo, estas plataformas distinguem-se do **SecDash** pelos seus objetivos e complexidade; focam-se habitualmente no segmento *en-*

terprise e na gestão operacional de ciclo completo, incluindo a orquestração de múltiplos *scanners* e fluxos de remediação (*remediation workflows*).

O SecDash, em contrapartida, não pretende ser uma ferramenta que efetua os seus próprios scans de segurança, mas sim focar-se especificamente na agregação, normalização e visualização centralizada de vulnerabilidades provenientes de múltiplos repositórios em plataformas distintas. O objetivo principal do projeto consiste em fornecer uma integração totalmente funcional com o GitHub e o GitLab, as duas plataformas mais amplamente utilizadas para hospedar código, uma vez que estas abrangem a vasta maioria dos casos de uso em ambientes reais. A integração com pipelines de Jenkins é considerada um *stretch goal*, a ser explorado caso a calendarização assim o permita.

1.2.1 DefectDojo

O **DefectDojo** é uma plataforma DevSecOps que atua como um agregador automático para o seu conjunto de ferramentas de segurança, permitindo organizar facilmente o trabalho de segurança e reportar a postura de segurança da organização aos *stakeholders*. [10]

De entre as funcionalidades do DefectDojo contam-se, entre outras:

- O rastreio e reporte de *Findings* de segurança;
- A imposição de SLAs (*Service Level Agreement*) em contexto;
- A gestão de *risk-acceptance* e outras decisões de triagem
- Coordenação com gestão de testes de penetração tradicionais;

Para além de possibilitar a integração com mais de 200 ferramentas de segurança, a sua instalação local requer múltiplos contentores para vários serviços, requerindo inclusive instâncias dedicadas de *Celery* e *Redis* enquanto *task-queue* para a gestão de tarefas assíncronas tais como o processamento de ficheiros de reporte volumosos ou a sincronização com o *Jira* e envio de notificações aos utilizadores. Estamos portanto perante uma infraestrutura comparativamente mais complexa que a requirida pelo SecDash, que será mais apropriada para equipas pequenas que não requeiram todas as funcionalidades disponíveis no DefectDojo.

Por outro lado, o modelo de dados utilizado pelo **DefectDojo** utiliza cinco classes distintas (***Product Types***, ***Products***, ***Engagements***, ***Tests*** e ***Findings***) para organizar o trabalho das equipas que o utilizem.[11] Um Engagement, por exemplo, representa um momento no tempo em que um *Test* é realizado, e contem um ou mais *Tests*. A necessidade de

instanciar um *Engagement* para cada nova análise introduz uma barreira de agilidade significativa. Apesar de se apresentar como flexível, de forma a que este modelo se possa adaptar às diferentes equipas que o utilizem, a abordagem do DefectDojo revela uma preocupação mais voltada para questões de auditoria do que para a visão imediata de vulnerabilidades e agilidade durante o processo de desenvolvimento do software.

1.2.2 ArmorCode

O **ArmorCode** é uma aplicação comercial *enterprise* de *Application Security Posture Management* (ASPM). que oferece integração com mais de 350 ferramentas de segurança de aplicações, infraestrutura, cloud e contentores, bem como com sistemas de DevOps.

De entre as suas vastas funcionalidades faz também parte a agregação de vulnerabilidades encontradas por outras ferramentas de *scanning*. De forma semelhante ao SecDash, o ArmorCode não faz ele próprio um scan de vulnerabilidades, apresentando-se como uma solução «*scannerless, vendor-agnostic*»[12]. Mais uma vez, no entanto, estamos perante uma ferramenta de elevada complexidade, possuindo imensas funcionalidades para lá do escopo do problema que o SecDash pretende resolver e que para além disso podem levar a uma curva de aprendizagem acentuada que poderá ser contraproducente quando se procura uma solução ágil para a simples agregação de relatórios de vulnerabilidades externos.

Quando comparado com o DefectDojo temos ainda a agravante de esta ser uma ferramenta SaaS proprietária com um modelo de licenciamento orientado para grandes corporações, o que a torna inacessível para projetos independentes ou de equipas de pequena dimensão.

1.2.3 ArcherySec

1.2.4 SecObserve

1.3 Requisitos Funcionais

Capítulo 2

Arquitetura e tecnologias

Este capítulo está organizado em duas secções onde se descreve o trabalho relacionado e alguns sistemas semelhantes ao sistema desenvolvido.

2.1 Visão geral do sistema

2.2 APIs externas

2.3 Tecnologias e linguagens

2.3.1 Backend

2.3.2 Frontend

Capítulo 3

Implementação

A nossa solução é apresentada neste capítulo. A solução consiste em grandes ideias, desenvolvidas e testadas.

Exemplo de indentação do segundo parágrafo.

3.1 Modelo de dados

3.2 Backend

3.3 Frontend

A avaliação de Projecto e Seminário envolve:

1. proposta do projeto;
2. relatório de progresso;
3. apresentação individual;
4. cartaz e versão beta do projeto;
5. relatório de projeto e discussão pública final.

A solução proposta assenta nas seguintes ideias. O algoritmo 1 apresenta as ações de pesquisa de um elemento E sobre um grafo G .

Algoritmo 1 Algoritmo de pesquisa em grafo.

Dados: Grafo G , Elemento E

Resultado: Localização de E em G

1. Para todos os vértices v em G
 2. Pesquisar e obter a localização de E
 - (a) Iniciar a lista de pontos, P
 - (b) Ordenar P
-

Nalgumas situações, é necessário apresentar excertos de código que ilustrem aspetos relevantes da implementação.

```
namespace ps;
public static void main() {
    System.out.println("PS - Projecto e Seminário");
}
```

Capítulo 4

Deployment

Este é o capítulo de testes. É possível forçar a inclusão de todas as referências com [].

Modo de matemática em texto $x = ma^2$ e em equação (duas formas):

$$x = ma^2$$

$$x = ma^2 \tag{4.1}$$

Capítulo 5

Dificuldades Técnicas e teóricas

Capítulo 6

Conclusão

Referências

- [1] D. L. Nguyen, D. H. Phan, T. D. Vu, and D. T. Ta, “Risk management in projects based on open-source software,” in *Proceedings of the 2019 8th International Conference on Software and Computer Applications (ICSCA '19)*, February 2019, pp. 178–183. [Online]. Available: <https://dl.acm.org/doi/10.1145/3316615.3316648>
- [2] CSIS — Center for Strategic and International Studies. (2024) Significant cyber incidents. Strategic Technologies Program. Acedido em: 04-03-2026. [Online]. Available: <https://www.csis.org/programs/strategic-technologies-program/significant-cyber-incidents>
- [3] CVE Program. (2014) CVE-2014-0160: Heartbleed vulnerability in openssl. MITRE Corporation. [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2014-0160>
- [4] National Institute of Standards and Technology. (2021) CVE-2021-44228: Apache log4j2 remote code execution vulnerability. National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>
- [5] L. H. Newman. (2021) The Log4Shell vulnerability is just the beginning. Conde Nast. [Online]. Available: <https://www.wired.com/story/log4j-log4shell-vulnerability-ransomware-second-wave/>
- [6] D. Goodin. (2021) As Log4Shell wreaks havoc, payroll service reports ransomware attack. Condé Nast. [Online]. Available: <https://arstechnica.com/information-technology/2021/12/as-log4shell-wreaks-havoc-payroll-service-reports-ransomware-attack/>
- [7] R. Lerman and C. Zakrzewski. (2021) The Log4j vulnerability has the internet on high alert. [Online]. Available: <https://www.washingtonpost.com/technology/2021/12/20/log4j-hack-vulnerability-java/>
- [8] United States Senate, “How Equifax failed to prevent the 2017 data breach,” Permanent Subcommittee on Investigations, Committee on Homeland Security and Governmental Affairs, Tech. Rep., 2019, staff Report. [Online]. Available: <https://nsarchive.gwu.edu/document/18370-national-security-archive-u-s-senate-permanent>

- [9] EPIC — Electronic Privacy Information Center. (2017) The Equifax data breach. Public Interest Research Center. [Online]. Available: <https://archive.epic.org/privacy/data-breach/equifax/>
- [10] DefectDojo Project. (2024) About DefectDojo: DevSecOps, ASOC, and Vulnerability Management. Official Documentation. [Online]. Available: https://docs.defectdojo.com/get_started/about/about_defectdojo/
- [11] ——. (2024) Product hierarchy: Product Types, Products, and Engagements. DefectDojo Documentation - Asset Modelling. [Online]. Available: https://docs.defectdojo.com/asset_modelling/hierarchy/product_hierarchy/
- [12] ArmorCode Inc. (2024) Unified vulnerability management: A modern approach. Official Platform Overview. [Online]. Available: <https://www.armorcode.com/unified-vulnerability-management>

Apêndice A

Exemplo de apêndice

Este é o primeiro parágrafo do apêndice.

Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.

Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

Nulla mattis luctus nulla. Duis commodo velit at leo. Aliquam vulputate magna et leo. Nam vestibulum ullamcorper leo. Vestibulum condimentum rutrum mauris. Donec id mauris. Morbi molestie justo et pede. Vivamus eget turpis sed nisl cursus tempor. Curabitur mollis sapien condimentum nunc. In wisi nisl, malesuada at, dignissim sit amet, lobortis in, odio. Aenean consequat arcu a ante. Pellentesque porta elit sit amet orci. Etiam at turpis nec elit ultricies imperdiet. Nulla facilisi. In hac habitasse platea dictumst. Suspendisse viverra aliquam risus. Nullam pede justo, molestie nonummy, scelerisque eu, facilisis vel, arcu.